

DocHub

1. Vision

Every product feature starts as a conversation. What happens after that conversation determines how well it gets built.

DocHub turns meeting transcripts into structured, context-aware PRDs — and then turns those PRDs into executable Jira tickets — without anyone spending hours on documentation or ticket creation.

More importantly, DocHub gets smarter with every PRD it processes. It builds a living knowledge base of the entire product, so every new document it creates is informed by everything that came before it. It knows what's been built, what's been discussed, and what still needs to be answered.

2. Problem Statement

Product teams lose critical context between meetings and execution. Transcripts sit unprocessed, PRDs are written in isolation without awareness of existing features, and user stories are created inconsistently — missing edge cases, validations, and clear acceptance criteria.

The opportunity is to make the path from conversation to execution intelligent, structured, and continuous.

3. Hackathon Goal

Build a working prototype that demonstrates this pipeline end to end:

Meeting transcript → context-aware PRD with open questions and risks → human reviews and downloads → edited PRD re-uploaded → vertically sliced user stories with acceptance criteria and validations → Jira tickets created automatically.

4. Scope

Must Have

- Transcript input via paste text and file upload (.txt / .docx)
- AI agent analyses transcript and generates a structured PRD in the defined format
- PRD includes an Open Questions / Risks section informed by the product knowledge base
- Generated PRD is downloadable as a branded .docx file
- User can re-upload an edited version of the PRD to continue the workflow
- Agent generates vertically sliced user stories from the PRD
- Each user story follows the defined format: Description, Acceptance Criteria, Validations table
- All stories displayed on UI before any action is taken
- "Create Jira Tickets" button pushes all stories to Jira in one action
- Persistent knowledge base — every processed PRD is stored and used as context for future generations

Out of Scope

- Inline editing of the PRD inside the tool
 - Authentication beyond simple name entry
 - Mobile responsiveness
 - Audio transcript input
 - Selecting specific PRD sections to generate stories for
 - Approvals workflow or role-based access
-

5. User Personas

The Product Manager Comes out of a feature discussion with a transcript or notes. Wants a structured PRD without writing it from scratch. Needs the output to reflect the full product context — not just what was said today. Will download, refine, and re-upload before generating stories.

The Engineering Lead Receives the user stories generated from the PRD. Needs stories that are independently deliverable, clearly scoped, and complete enough to start development without follow-up questions. Values precise acceptance criteria and well-defined validations.

6. End-to-End User Flow

None

1. User opens the app and enters their name
 2. User submits a meeting transcript
 - Option A: Paste raw text into the input box
 - Option B: Upload a .txt or .docx file
 3. User clicks "Analyse & Generate"
 4. Agent queries the knowledge base for existing product context
 5. Agent generates a structured PRD:
 - Populates all 6 standard sections from the transcript
 - Adds an Open Questions / Risks section from knowledge base analysis
 6. PRD is displayed in a preview with company branding
 7. User downloads the PRD as a .docx file
 8. User makes edits offline based on stakeholder feedback
 9. User re-uploads the edited .docx file
 10. User clicks "Generate User Stories"
 11. Agent reads the final PRD and generates all user stories
 12. All stories are displayed on app screen – user reviews them
 13. User clicks "Create Jira Tickets"
 14. All stories are pushed to Jira simultaneously
 15. Confirmation screen shows all created tickets with direct Jira links
 16. PRD is saved to the knowledge base for future context
-

7. PRD Output Format

Every generated PRD must follow this structure. The agent must populate all sections — inferring intelligently where the transcript is incomplete and clearly marking any inferred content for the reviewer.

Section	What It Contains
Project Title	A clear, meaningful name for the feature set (e.g. "StayFit+ Initiative")
Description	Brief overview of what is being proposed
Problem	The specific problem this feature is solving
Why	Evidence or signals that confirm this is a real, validated problem
Success	Measurable outcomes that confirm the problem has been solved
Audience	The clearly defined user segment this is being built for
Open Questions / Risks	Conflicts, gaps, and unresolved questions identified by the agent — see Section 8

Handling Incomplete Transcripts

- The agent must never leave a section blank
 - Where information is missing, the agent makes a clearly labelled inference
 - Inferred content must be visually distinguishable in the preview so the reviewer knows what to validate
 - The agent must never fabricate specific data points, metrics, or research findings not implied by the transcript
-

8. Agent Behaviour — Knowledge Base & Open Questions

Why This Matters

This is what separates DocHub from a simple document generator. The agent doesn't just read today's transcript — it reads it in the context of the entire product history. Every PRD it has ever processed becomes part of its understanding of the product.

Agentic AI & RAG Architecture

The agent uses a **Retrieval-Augmented Generation (RAG)** approach:

- All previously generated and approved PRDs are stored in a vector knowledge base
- When a new transcript is submitted, the agent retrieves the most relevant past PRDs using semantic search
- The retrieved context is passed into the generation prompt alongside the new transcript
- The agent reasons across both the new input and the retrieved history to produce a grounded, consistent PRD

This means the agent is not working from memory — it is actively retrieving and reasoning over real product documents every time it generates.

Open Questions / Risks Section

This section must be generated by the agent after querying the knowledge base. It should contain two types of entries:

Type 1 — Conflicts & Overlaps Where the new PRD proposes something that intersects with an existing feature or a past decision.

Example: "The proposed notification system overlaps with the alert mechanism defined in the Push Notifications PRD (March 2025). Clarify whether this is an extension or a replacement."

Type 2 — Gaps from the Transcript Where the transcript did not address something that a complete PRD should cover.

Example: "Pricing model for premium users was not discussed. This may affect the Audience and Success sections." Example: "Error handling for failed uploads was not mentioned. Engineering will need this defined before development begins."

Both types must be present in every PRD where applicable. If no conflicts are found and no gaps exist, the section should say so explicitly — it must never be omitted.

Knowledge Base Growth

- Every PRD that completes the full workflow (transcript → stories → Jira) is saved to the knowledge base
 - The knowledge base must be pre-seeded with 2–3 realistic PRDs before the demo so context-awareness is demonstrable on the day
 - The knowledge base is persistent across sessions
-

9. User Story Format

Every user story generated by the agent must follow this exact structure.

Title: [Short, scannable name for the story]

Description: As a [user type], I want to [action], so that [outcome].

Acceptance Criteria:

- [Criterion 1 — a specific, testable statement of behaviour]
- [Criterion 2]
- [Criterion 3]
- ...

Validations:

Validation	Response
[Input, action, or condition being tested]	[Expected system behaviour or message]
[Edge case or error state]	[Expected system behaviour or message]
...	...

Vertical Slicing Rules

The agent must slice stories so that each one delivers end-to-end value independently. A story must not be purely frontend, purely backend, or purely a component — it must represent a complete, shippable unit of user value.

The agent decides slice boundaries autonomously based on the PRD content. As a guide:

- Each distinct user action described in the PRD should produce at least one story
- Stories should not be so large that they cannot be completed in a single sprint
- Stories should not be so small that they have no standalone value

Acceptance Criteria Rules

- Written as testable statements, not vague descriptions
- Must cover the happy path, at minimum one alternative path, and at minimum one error state
- Must be specific enough that a developer can implement and a QA can verify without asking follow-up questions

Validation Table Rules

- Every meaningful input, state, or edge case for the story gets its own row
 - The Response column describes exactly what the system does — including UI messages where relevant
 - Rows must cover: valid inputs, invalid inputs, empty/missing inputs, boundary conditions, and system error states where applicable
-

10. Jira Integration

When the user clicks "Create User Stories":

- All displayed user stories are pushed to Jira simultaneously in a single action
- Each story becomes one Jira ticket containing: Title, Description (user story), Acceptance Criteria, and Validations table
- The structure of tickets within Jira (flat vs. Epic hierarchy) is left to the team's discretion based on Jira project configuration
- A confirmation screen displays all created tickets with their Jira IDs and direct links
- The push must be atomic — either all tickets are created or the user is shown a clear error. Partial creation without notification is not acceptable

Pre-event dependency: Jira project key, API token, and base URL must be configured and tested before the hackathon begins. This cannot be set up on the day.

11. Success Criteria

The prototype is successful if a judge can watch the following happen live without failure:

The prototype is successful if a judge can watch the following happen live without failure:

- User submits a transcript via paste and the PRD is generated
 - All 7 PRD sections are populated including Open Questions / Risks
 - At least one knowledge base insight is visible in the Open Questions section
 - PRD preview is readable and branded correctly
 - PRD downloads as a properly formatted .docx file
 - User re-uploads an edited version and proceeds to story generation
 - User stories are generated — each with Description, Acceptance Criteria, and Validations table
 - Stories are visibly vertically sliced
 - "Create Jira Tickets" push succeeds and confirmation screen shows ticket links
 - Full flow completes in under 10 minutes
-

Scope is frozen at this document. Anything not listed here goes on a post-hackathon backlog.